11
Design A Notification System

A notification system has already become a very popular feature for many applications in recent years. A notification alerts a user with important information like breaking news, product updates, events offerings, etc. It can be sent as a text message, push notification, or an in-app alert. In this chapter, you are asked to design a notification system.

A notification is more than just mobile push notifications. Three types of notification formats are mobile push notification, SMS message, and Email. Figure 1 shows an example of each of these notifications.

Figure 1

Step 1 - Understand the problem and establish design scope

Building a scalable system that sends out billions of notifications a day is not an easy task. It requires a deep understanding of the notification ecosystem. The notification ecosystem is sometimes referred to the system and subsystems, and it is your responsibility to ask questions to clarify the requirements.

Candidate: What types of notifications does the system support?
Interviewer: Push notification, SMS message, and email.
Candidate: What is the scale of the system?
Interviewer: Let's say it is a soft real-time system. We want a user to receive notifications as soon as possible. However, if the system is under a high workload, a slight delay is acceptable.
Candidate: What are the supported devices?
Interviewer: iOS devices, android devices, and laptop/desktop.
Candidate: What triggers notifications?
Interviewer: Notifications can be triggered by client applications. They can also be scheduled on the server side.
Candidate: Will users be able to opt out?
Interviewer: Yes, users who choose to opt out will no longer receive notifications.
Candidate: How many notifications are sent out each day?
Interviewer: 10 million mobile push notifications, 1 million SMS messages, and 5 million emails.

Step 2 - Propose high-level design and get buy-in

This section shows the high-level design that supports various notification types: iOS push notification, Android push notification, SMS message, and Email. It is structured as follows:

- Different types of notifications.
- Contact info gathering flow.
- Notification sending/receiving flow.

Different types of notifications

We start by looking at iOS push notification system as a high level.

iOS push notification

Figure 2

We primarily need three components to send an iOS push notification:

- Provider. A provider builds and sends notification requests to Apple Push Notification Service (APNS). To construct a push notification, the provider provides the following data:
  - Device token. This is a unique identifier used for sending push notifications.
  - Payload. This is a JSON dictionary that contains a notification's payload. Here is an example:

```
{
  "aps" : {
    "alert" : "Game Request",
    "badge" : 5,
    "category" : "GAME_INVITE",
    "sound" : "bingo.aiff",
    "mutable-content" : 1
  },
  "gameID" : "12345678"
}
```

- APNS. This is a remote service provided by Apple to propagate push notifications to iOS devices.
- iOS Device. It is the end device which receives push notifications.

Android push notification

Android adopts a similar notification flow. Instead of using APNs, Firebase Cloud Messaging (FCM) is commonly used to send push notifications to Android devices.

Figure 3

SMS message

For SMS messages, third party SMS services like Twilio [1], Nexmo [2] and Inday others are commonly used. Most of them are commercial services.

Figure 4

Email

Although companies can set up their own email servers, many of them opt for commercial email services. Sendgrid [3] and Mailchimp [4] are among the most popular email services. APNS offers customer support and analytics.

Figure 5

Figure 6 shows the design after including all the third party services.

Figure 6

Contact info gathering flow

To send notifications, we need mobile device tokens, phone numbers, or email addresses. As shown in Figure 7, when a user installs our app or signs up for the first time, API servers collect user contact info and store it in the database.

Figure 7

Figure 8 shows simplified database tables to store contact info. Email addresses and phone numbers are stored in the user table, whereas device tokens are stored in a new table since a user may have multiple devices, indicating that a push notification can be sent to all the user devices.

Figure 8

Notification sending/receiving flow

We will first present the initial design, then, propose some optimizations.

High level design

Figure 9 shows the design: end-each system component is explained below.

Figure 9

Service 1 to N. A service can be a micro service, a cron job, or a distributed system that triggers notification sending events. For example, a billing service may send payment reminders, a shopping cart may send an email to remind shoppers about their purchases, and a social network may send various notifications.

Notification system. The notification system is the centerpiece of sending/receiving notifications. Starting with something simple, only one notification server is used to handle all the flows. As the system is built, and mobile notification propagation for third-party services.

Third party services. Third party services are responsible for delivering notifications to users. While integrating with third-party services, we need to be flexible as we may want to switch to different third-party services in the future, or we may want to integrate with new third-party services. Another important consideration is how many third-party service vendors support our customers in the future. For instance, if APNS is unavailable in China, then alternative third-party services such as Jpush, PushM, etc. are used there.

iOS, Android, SMS, Email. These are various notifications on their devices.

These problems are identified in this design:

- A single point of failure (SPOF) in single notification server means SPOF.
- Hard to scale. The notification system handles everything related to push notifications in one server. It is challenging to scale databases, cache, message queues, and various processing components.
- Performance bottleneck. Processing and sending notifications, and receiving notifications from third-party services are resource intensive, waiting for responses from third party services could take time. Handling everything in one system can result in the system overload, especially during peak hours.

High level design (improved)

When incorporating the changes in the initial design, we improve the design as listed below:

- Move the database and cache out of the notification server.
- Add notification servers and allow for horizontal scaling of notification servers.
- Introduce message queues to decouple the system components.

Figure 10 shows the improved high level design.

Figure 10

Service 1 to N. Unchanged from the previous design.

Service Load Balancer. Provides load balancing to API servers.

Notification Servers. The notification system is the centerpiece of sending/receiving notifications. Starting with something simple, only one notification server is used to handle all the flows. As the system is built, and mobile notification propagation for third-party services.

Third Party Servers. Third party services are responsible for delivering notifications to users. While integrating with third-party services, we need to be flexible as we may want to switch to different third-party services in the future, or we may want to integrate with new third-party services. Another important consideration is how many third-party service vendors support our customers in the future. For instance, if APNS is unavailable in China, then alternative third-party services such as Jpush, PushM, etc. are used there.

iOS, Android, SMS, Email. These are various notifications on their devices.

These problems are identified in this design:

- A single point of failure (SPOF) in single notification server means SPOF.
- Hard to scale. The notification system handles everything related to push notifications in one server. It is challenging to scale databases, cache, message queues, and various processing components.
- Performance bottleneck. Processing and sending notifications, and receiving notifications from third-party services are resource intensive, waiting for responses from third party services could take time. Handling everything in one system can result in the system overload, especially during peak hours.

High level design (improved)

When incorporating the changes in the initial design, we improve the design as listed below:

- Move the database and cache out of the notification server.
- Add notification servers and allow for horizontal scaling of notification servers.
- Introduce message queues to decouple the system components.

Figure 10 shows the improved high level design.

Figure 10

Service 1 to N. Unchanged from the previous design.

Service Load Balancer. Provides load balancing to API servers.

Notification Servers. They provides the following functionalities:

- Provide APIs for services to send notifications. Those APIs will be only accessible internally or by verified clients to prevent spamming.
- Carry out basic validations to verify emails, phone numbers, etc.
- Query the cache and database to fetch data needed to compose a notification.
- Put notification data to message queues for parallel processing.

Here is an example of the API to send an email:

POST https://api.sendgrid.com/v3/mail/send

Request body:

```
{
    "personalizations": [
        {
            "to": [
                {
                    "email": "john.doe@example.com"
                }
            ],
            "subject": "Hello, World!"
        }
    ],
    "from": {
        "email": "from_address@example.com"
    },
    "content": [
        {
            "type": "text/plain",
            "value": "Hello, World!"
        }
    ]
}
```

Cache. User info, device info, notification templates are cached.

DB. It stores data about user, notification settings, etc.

Message queues. They remove dependencies between components. Message queues serve a buffer when high volumes of notifications are to be sent out. Each notification type is assigned with a distinct message queue so an outage in one third party service will not affect other notification streams.

Workers. Workers are a list of servers that pull notification events from message queues and send them to the corresponding third-party services.

Third party services. Already explained in the initial design.

iOS, Android, SMS, Email. Already explained in the initial design.

Here are a sample list of events that could happen in the system:

1. Service 1 and service N send notification events to message queues.
2. Notification server 1 pulls notification events from message queues and sends notification data to third party services.
3. Third party services notify a user's device.

Notification server 1 pulls events from message queues.
Notification server 1 fetches user info and device info from cache or database.
Notification server 1 sends the corresponding payload for processing. For instance, an iOS push notification event is sent to the iOS Push system.
Workers pull notification events from message queues.
Workers send notifications to third-party servers.
Third party servers send notifications to user devices.

Step 3 - Design deep dive

In the high level design we discussed different types of notifications, contact info gathering flow, and notification sending/receiving flow. We will explore more components in the following:

- Reliability.
- Additional component and consideration: notification template, notification settings, rate limiting, retry mechanism, security in push notifications, monitor queued notifications and event tracking.
- Updated design.

Reliability

How to prevent data loss?

One of the most important requirements in a notification system is that it cannot lose data. Notifications can usually be delayed or in a limited loss even but. To satisfy this requirement, the notification system persists notification data in a database and implements a very mechanism. The notification log database is created for data persistence, as shown in Figure 11.

Figure 11

Will recipients receive a notification exactly once?

The short answer is no. Although notification is delivered exactly once most of the time, the distributed nature could result in duplicate notifications. To reduce the duplication occurrence, we introduce a dedupe mechanism and handle each failure case carefully. Here is a simple design:

After a notification event first arrives, we check if it is seen before by checking the event ID. If it is seen before, it is discarded. Otherwise, we will send out the notification. The mechanism needed to implement this will include event (exactly once) delivery, refer to the reference material [5].

Additional components and considerations

We have discussed how to collect user contact info, send and receive a notification. A notification system is a lot more than that. Here we discuss additional components including template tracking, notification settings, event tracking, system monitoring, rate limiting, etc.

Notification template

A large notification system sends out millions of notifications per day, and many of these notifications follow a similar format. Notification templates are introduced to avoid building every notification from scratch. A notification template is a preformatted notification to create your unique notification by specifying parameters, styling, tracking links, etc. Here is an example template of push notifications:

```
Text template of iOS, Android and SMS:
BODY: Hello, [name], [item] [action] is back! Hurry up, only a few left! [CTA]

Email template:
BODY: Hello, [name], [item] [action] is back! Hurry up, only a few left! [CTA]
```

The benefits of using notification templates include maintaining a consistent format, reducing the margin error, and saving time.

Notification settings

Users generally create opt-ins to receive notifications daily and they can easily feel overwhelmed. Thus, many solutions and apps give users fine grained control over notifications. The information is stored in the notification setting table with the following fields:

```
user id, bigint
channel number of push notification, email or SMS
opt_in Boolean if user is to receive notification
```

Before any notification is sent to a user, we first check if a user is opted in to receive this type of notification.

Rate limiting

To avoid overwhelming users with too many notifications, we can limit the number of notifications a user can receive. This is important because recipients could turn off notifications completely if they are annoyed.

Retry mechanism

When a third party service fails to send a notification, we add it to the queue for message queue for retrying. If the problem persists, an alert will be sent out to developers.

Security in push notifications

iOS or Android apps, appkey and appsecret are used to secure push notification APIs. [6]. Only authenticated or verified clients are allowed to send push notifications using our APIs. Advanced authentication is beyond the reference material [6].

Monitor queued notifications

A big metric to monitor is the total number of queued notifications. If the number is large, the notification events are not processed fast enough by systems. To avoid delay in the notification delivery, more workers are needed. Figure 12 (visits to CD) shows an example of queued messages to be processed.

Figure 12

Events tracking

Notification metrics, such as open rate, click rate, and engagement are important in understanding customer behaviors. Analytics service implements events tracking. Integration between the notification system and the analytics service is usually required. Figure 13 shows an example of events that might be tracked for analytics purposes.

Figure 13

Updated design

Putting everything together, Figure 14 shows the final architecture system design.

Figure 14

In this design, many new components are added in comparison with the previous design.

- The notification servers are equipped with two more critical features: authentication and rate limiting.
- Two new components are added to improve extensibility: cache and template. Cache is used to store notification templates. We also put both in the messaging queue and the worker will pull data for a predefined number of times.
- Furthermore, notification templates provide a consistent and efficient notification creation process.
- Finally, monitoring and tracking systems are added for system health status and future discussions.

Step 4 - Wrap up

Notifications are complicated because they keep a growth with important information. It could be a push notification about your favorite movie on Netflix, an email about discounts on new products, or a message about your online shopping payment confirmation.

In this chapter, we diversified the design of a scalable notification system that supports various notification formats: push notifications, SMS messages and email. We adopted message queues to decouple system components.

Besides the high level design, we dug deep into more components and optimizations:

- Reliability. We proposed a dedupe entry mechanism to minimize the failure rate.
- Extensibility. We introduced notification templates, integration with third party notifications.
- Tracking and monitoring. These are implemented in any stage of a notification flow to capture important data.
- Respect user settings. Users may opt-out of receiving notifications. Our system checks user settings first before sending notifications.
- Rate limiting. Users are annoyed by receiving too many notifications.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] Twilio SMS: https://www.twilio.com/sms

[2] Nexmo SMS: https://www.nexmo.com/products/sms

[3] Sendgrid: https://sendgrid.com/

[4] Mailchimp: https://mailchimp.com/

[5] You Cannot Have Exactly-Once Delivery: https://bravenewgeek.com/you-cannot-have-exactly-once-delivery/

[6] Security in Push Notifications: https://www.airship.com/resources/white-paper/security-in-push-notifications-on-ios-and-android/

[7] Key metrics for Notifications monitoring: https://www.datadoghq.com/blog/monitoring-sre-metrics/